R Basics

Fundamentals of Computing and Data Display

9/4/2024

GitHub

- Part 1 of Assignment 1 requires using Git and GitHub with others. If you need a group/partner, please let me know!

- If you are having issues with using GitHub, let me know ASAP!

Working with R and RStudio

- Work in a Git repository and track your work.
- Keep your files organized!
- Use Rmarkdown/Quarto files (will be covered next week).
  - This will help organize the information that you need in your R code files, such as file headers, comments, text along with code, etc.

- RStudio can help you manage all of this.

Reading Files

- Easiest way to check type of separator and other aspects of a data file before reading them is to check outside of R.
  - Codebooks and code documentation can also be helpful for this.
- If the file is too big, then we likely want to use a database and SQL anyway (will be covered later in this course).

Troubleshooting

- Common mistakes:
  - Typos!
  - Objects are the wrong class
    - Numeric vs. character
    - Lists vs. vectors vs. data frames

- Steps to take to troubleshoot code:
  - Check to make sure you haven’t made a typo or misspelling!
  - Check the class of an object.
  - Check the help file using “?” to see if the function does what you think it should.

Updating Packages

- When to use update.packages()?
  - Sometimes, it’s actually disadvantageous to update certain packages because you might break some dependencies.
  - Recommend doing it somewhat regularly to keep up to date, but it’s not a big deal if there are long gaps.
  - You might need to update packages (or go backwards) when installing new packages.

Types (or modes or classes) of Objects

- `typeof` vs `mode` vs `class`
  - Many times, these will have similar usages.
  - `class` will usually be more different compared to `typeof`/`mode`
  - In practice, using any one should be enough for your purpose of determining what type of object it is.
  - RStudio also has a useful pane to give you this information

Loops

- Loops are generally slow … but they are not all bad! Sometimes, they are unavoidable.
- Processes that can be vectorized:
  - Using functions with a range of different arguments
  - Do something with all rows or columns of a data frame
- Processes that might use loops:
  - Simulations
  - Sampling

Functions

- How to identify when to use functions:
  - 1) **You are doing a "generic" process that you will need to do many times**. This is the classic case of using a function to avoid having to copy and paste code.
  - 2) **You're using a loop of some sort.** Sometimes, loops can be short, but it can be easier to follow a loop if you include functions to break up long code within the loop.
  - 3**) It makes the code easier to follow**. There might be sequences of code that can be difficult to follow without existing functions. For example, if you want to sample, then calculate a statistic, then create a visualization, it might be helpful to put each step (or parts of steps) into a function.

Building Functions

- One process for building functions:
  - **Step 1**: Write the code for one use case. You can separate out pieces as variables to be changed, or just hard code for now.
  - **Step 2**: Think about what you would need to change to make sure that it is generalizable to other situations. What parts of the code would change if applied to a new scenario?
  - **Step 3**: Change the code for one use case to become more generalized and create arguments to match.

- Test your function as you go! Build your function one by one, not all at once!